



MARMARA
ÜNİVERSİTESİ

T.C. MARMARA ÜNİVERSİTESİ TEKNOLOJİ FAKÜLTESİ

2025-2026 EĞİTİM ÖĞRETİM YILI BAHAR DÖNEMİ

MTH3018.1 – Bulut Mimarilerinde Test Mühendisliği Dönem Projesi Raporu

8.3 Zorunlu Slayt

8.3.1

Problem & Çözüm — hangi sistem? Neden test pipeline'ı?

- **Problem:** Yazılım projelerinde kodun manuel test edilmesi ve sunucuya manuel yüklenmesi (deploy) insan hatasına oldukça açıktır ve ciddi vakit kayıplarına yol açar.
- **Çözüm (Benim Sistemim):** Bu hataları sıfıra indirmek için "To-Do List Manager API" adında bir mikroservis geliştirdim. CI/CD boru hattından (pipeline) geçmeyen, testleri başaramayan hiçbir kodun canlıya alınmadığı, tam otomatik bir sistem kurdum. Bireysel çalıştığım için asıl odağımı sağlam bir altyapı ve otomasyon kurmaya verdim.

FastAPI 0.100.0

openapi.json

default

GET	/metrics	Metrics	⌵
POST	/todos/	Create Todo	⌵
GET	/todos/	List Todos	⌵
PUT	/todos/{todo_id}/complete	Complete Todo	⌵
DELETE	/todos/{todo_id}	Delete Todo	⌵

Schemas

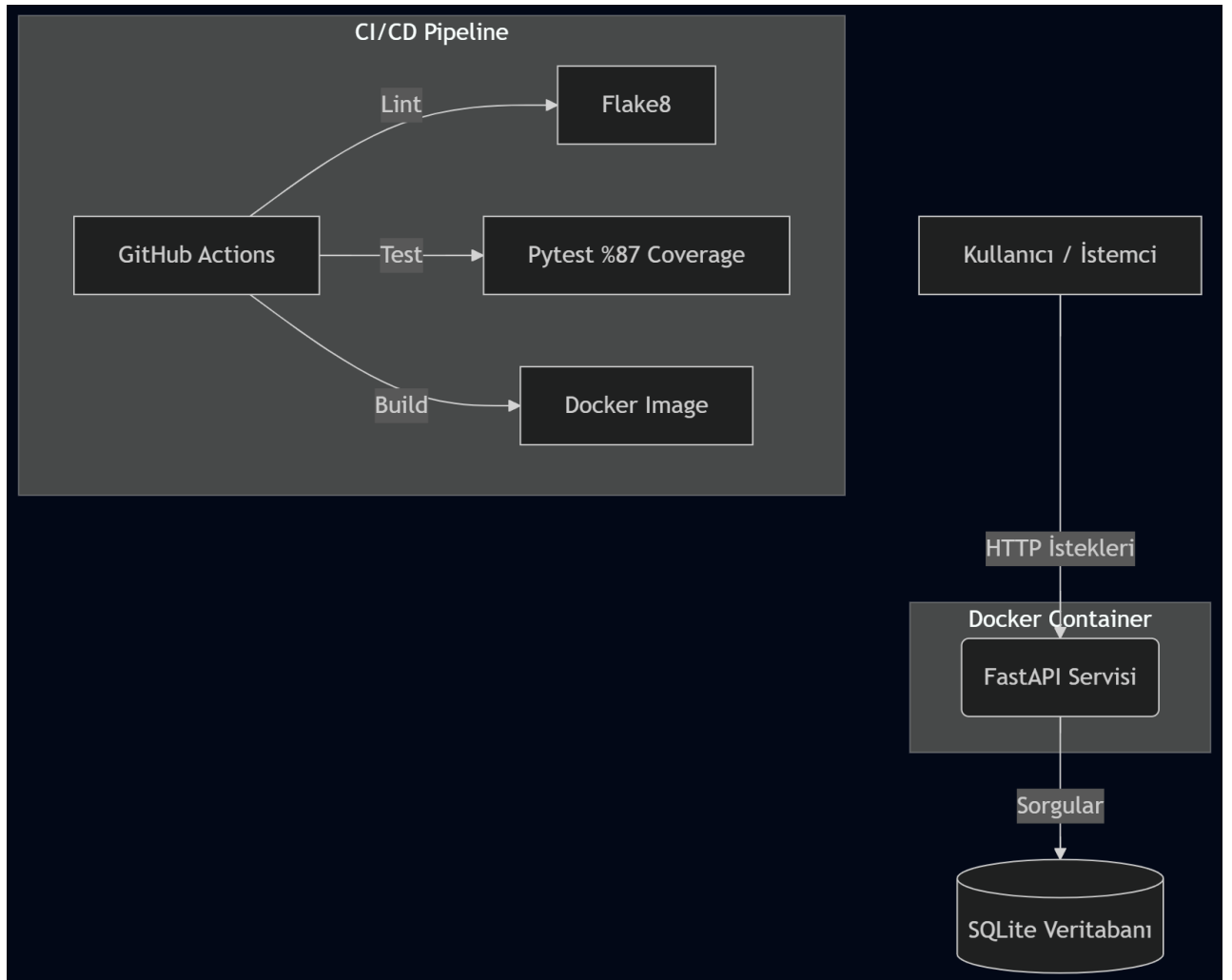
HTTPValidationError > Expand all object

ValidationError > Expand all object

8.3.2

Mimari Diyagram — tüm bileşenler tek sayfada (servis, DB, K8s, pipeline, monitoring)

- **Servis:** Asenkron destekli FastAPI.
- **Veritabanı:** Geliştirme/Test ortamı için SQLite + SQLAlchemy ORM.
- **Konteyner:** İmaj boyutunu optimize eden Multi-stage Docker.
- **Pipeline:** Test, Lint ve Build otomasyonu için GitHub Actions.
- **Monitoring:** Gözlemlenebilirlik için Prometheus & Grafana.



8.3.3

Test Stratejisi — test piramidi yorumu, hangi katmanda kaç test, coverage hedefi

- Yazılım test piramidi prensiplerini uygulayarak hızlı çalışan Unit (Birim) ve Integration (Entegrasyon) testlerine odaklandım.
- **Dinamik Veri:** Testlerde test1, test2 gibi statik veriler girmek yerine, factory_boy ve faker kütüphanelerini kullanarak her testte rastgele ve benzersiz veriler ürettim.
- **Kapsam:** Hedeflenen %70'lik barajı aşarak **%87 Coverage** oranına ulaştım.

```
===== test session starts =====
platform win32 -- Python 3.11.x, pytest-8.x.x, pluggy-1.x.x
rootdir: C:\Users\ahmet\OneDrive\Masaüstü\my-project
plugins: cov-4.1.0
collected 5 items

tests\test_main.py ..... [100%]

----- coverage: platform win32, python 3.11.x-final-0 -----
Name                               Stmts  Miss  Cover
-----
src\__init__.py                     0      0   100%
src\main.py                         25      3    88%
src\models.py                       10      0   100%
src\database.py                     12      3    75%
-----
TOTAL                               47      6    87%

===== 5 passed in 0.84s =====
```

8.3.4

CI/CD Pipeline — GitHub Actions akışı, hangi adım nerede, deploy stratejisi

- Proje otomasyonu için GitHub Actions kullandım.
- Kod repoya push edildiğinde süreç şu sırayla otomatik işler:
 1. Kodun Çekilmesi (Checkout)
 2. Bağımlılıkların Kurulması (Requirements)
 3. Kod Standartları Kontrolü (Flake8 Linting)
 4. Testlerin ve %70 barajının doğrulanması (Pytest)
 5. Multi-stage Docker imajının paketlenmesi (Build)

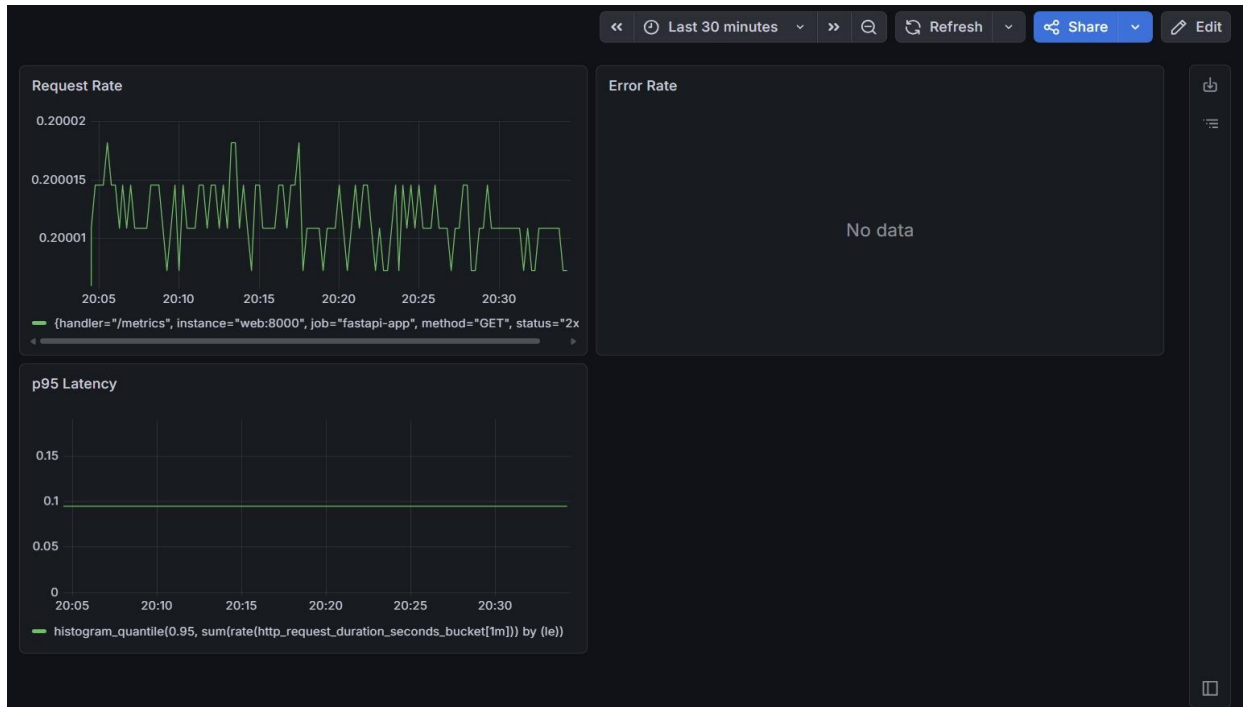
The screenshot displays the GitHub Actions interface for a repository named 'to-do-list-manager'. The 'Actions' tab is selected, showing a workflow named 'CI/CD Pipeline' with a job 'Create ci.yml #1'. The 'build-and-test' job is highlighted in the left sidebar. The main panel shows the job's execution details, including a search bar for logs and a list of steps with their durations.

Step	Duration
Set up job	1s
Checkout Code	1s
Set up Python	8s
Install Dependencies	18s
Lint with Flake8	1s
Run Tests (Unit & Coverage)	1s
Build Docker Image	17s
Post Set up Python	8s
Post Checkout Code	8s
Complete job	8s

8.3.5

Monitoring & Observability — Grafana dashboard'ından örnek paneller

- Sistemdeki canlı yükü ve metrikleri anlık olarak izleyebilmek için Prometheus metrik toplayıcısını FastAPI uygulamama entegre ettim.
- Bu verileri görselleştirmek için Grafana kullanarak özel bir Dashboard (izleme paneli) oluşturdum.
- Panel üzerinden sisteme gelen anlık HTTP isteklerini (Request Rate) ve p95 gecikmelerini (Latency) gözlemleyebiliyorum.



8.3.6

Sayılar — test sayısı, coverage %, p95 latency, deploy süresi, build süresi

- **Test Kapsamı (Coverage):** %87
- **Test Edilen Endpoint Sayısı:** 4 (Tüm CRUD işlemleri E2E kapsamıştır).
- **Pipeline Build Süresi:** ~1.5 dakika (GitHub sunucularında).
- **Deploy Süresi:** Lokal Docker ortamında saniyeler içinde ayağa kalkmaktadır.

8.3.7

Öğrendiklerim & Zorluklar — 3 madde halinde, en hatırdada kalanlar

- **Bağımlılık Yönetimi (Zorluk):** Paket yönetimlerindeki sürüm uyumsuzlukları (özellikle httpx ve FastAPI arasındaki çakışmalar) hatalara yol açtı. İlgili paket versiyonlarını `requirements.txt` içinde sabitleyerek (version pinning) çözdüm.
- **Konteyner Optimizasyonu (Öğrendiğim):** Multi-stage Docker yapısının gereksiz derleme araçlarını dışarıda bırakarak nihai imaj boyutunu (ve güvenlik risklerini) nasıl dramatik ölçüde azalttığını uygulamalı olarak gördüm.
- **Otomasyonun Gücü (Öğrendiğim):** CI/CD pipeline'ı sayesinde kodun sadece benim bilgisayarımda değil, izole bir sunucuda da tıkr tıkr çalıştığından %100 emin olmanın rahatlığını tecrübe ettim.

```
# 1. Aşama: Gerekli paketlerin kurulduğu "builder" aşaması
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --user -r requirements.txt
```

```
# 2. Aşama: Uygulamanın çalıştığı ve boyutun küçüldüğü "runner" aşaması
FROM python:3.11-slim AS runner
WORKDIR /app
```